



Prediction API Overview

- This document lists the base URL for the API endpoints: **<https://papi.asterdex.com>**
- All API responses are in JSON format.

API Agent Address settings

- Many endpoints require an API Key to access.
- When setting the Agent Address, for security reasons it is recommended to set an IP access whitelist.
- **Never reveal your Agent Address/private key to anyone.**

If an API Key is accidentally exposed, immediately delete that Key and generate a new one.

Attention

- TESTUSDT or any other symbols starting with TEST are symbols used for Aster's INTERNAL TESTING ONLY. Please DO NOT trade on these symbols starting with TEST. Aster does not hold any accountability for loss of funds due to trading on these symbols. However, if you run into issues, you may contact support about this any time, we will try to help you recover your funds.

V3 Nonce Mechanism

- **Nonce** is used to validate the **validity, uniqueness, and replay-protection** of requests. Clients should use the **current timestamp (microsecond precision)** as the nonce, and the difference from server time must not exceed **10 seconds**.
- Request processing flow:
 - i. If the nonce **has already been used** → rejected as a **duplicate request**
 - ii. Otherwise, the system checks whether it is **too old**
- To improve performance, each user maintains only the **most recent 100 nonces**:
 - If the list is full and the new nonce is **smaller than the current minimum** → rejected as **expired**
 - Otherwise, the **oldest nonce is removed** and the new one is added
- Overall effect: **prevents duplicates and stale requests, retaining only recent valid**

requests

HTTP return codes

- HTTP `4XX` status codes are used to indicate errors in the request content, behavior, or format. The problem lies with the requester.
- HTTP `403` status code indicates a violation of WAF restrictions (Web Application Firewall).
- HTTP `429` error code indicates a warning that the access frequency limit has been exceeded and the IP is about to be blocked.
- HTTP `418` indicates that after receiving a 429 you continued to access, so the IP has been blocked.
- HTTP `5XX` error codes are used to indicate issues on the Aster service side.

API error codes

- When using the endpoint `/api/v3`, any endpoint may throw exceptions;

The API error codes are returned in the following format:

```
{  
  "code": -1121,  
  "msg": "Invalid symbol."  
}
```

Basic information about the endpoint

- Endpoints with the `GET` method must send parameters in the `query string`.
 - For `POST`, `PUT`, and `DELETE` endpoints, parameters can be sent in the `query string` with content type `application/x-www-form-urlencoded`, or in the `request body`.
 - The order of parameters is not required.
-

Access restrictions

Basic information on access restrictions

- The `rateLimits` array in `/api/v3/exchangeInfo` contains objects related to `REQUEST_WEIGHT` and `ORDERS` rate limits for trading. These are further defined in the `enum definitions` section under `rateLimitType`.
- A 429 will be returned when any of the rate limits are violated.

IP access limits

- Each request will include a header named `X-MBX-USED-WEIGHT-(intervalNum)(intervalLetter)` that contains the used weight of all requests from the current IP.
- Each endpoint has a corresponding weight, and some endpoints may have different weights depending on their parameters. The more resources an endpoint consumes, the higher its weight will be.
- Upon receiving a 429, you are responsible for stopping requests and must not abuse the API.
- **If you continue to violate access limits after receiving a 429, your IP will be banned and you will receive a 418 error code.**
- Repeated violations of the limits will result in progressively longer bans, **from a minimum of 2 minutes up to a maximum of 3 days.**
- The `Retry-After` header will be sent with responses bearing 418 or 429, and will give the wait time **in seconds** (if 429) to avoid the ban, or, if 418, until the ban ends.
- **Access restrictions are based on IP, not API Key**

You are advised to use WebSocket messages to obtain the corresponding data as much as possible to reduce the load and rate-limit pressure from requests.

Order rate limits

- Each successful order response will include a `X-MBX-ORDER-COUNT-(intervalNum)(intervalLetter)` header containing the number of order limit units currently used by the account.
- When the number of orders exceeds the limit, you will receive a response with status 429

but without the `Retry-After` header. Please check the order rate limits in `GET api/v3/exchangeInfo` (`rateLimitType = ORDERS`) and wait until the ban period ends.

- Rejected or unsuccessful orders are not guaranteed to include the above header in the response.
- **Order placement rate limits are counted per account.**

WebSocket connection limits

- The WebSocket server accepts a maximum of 5 messages per second. Messages include:
 - PING frame
 - PONG frame
 - Messages in JSON format, such as subscribe and unsubscribe.
 - If a user sends messages that exceed the limit, the connection will be terminated. IPs that are repeatedly disconnected may be blocked by the server.
 - A single connection can subscribe to up to **1024** Streams.
-

API authentication types

- Each API has its own authentication type, which determines what kind of authentication should be performed when accessing it.
- The authentication type will be indicated next to each endpoint name in this document; if not specifically stated, it defaults to `NONE`.

Security Type	Description
NONE	API that does not require authentication
PREDICTION_TRADE	A valid signer and signature are required
USER_DATA	A valid signer and signature are required
USER_STREAM	A valid signer and signature are required
MARKET_DATA	API that does not require authentication

SIGNED (TRADE AND USER_DATA) Endpoint security

- Security Type: PREDICTION_TRADE, USER_DATA, USER_STREAM

Example of POST /api/v3/order

The following parameters are API registration details. The values for user, signer, and privateKey are for demonstration purposes only (the privateKey corresponds to the signer).

Key	Value	De
user	0x63DD5aCC6b1aa0f563956C0e534DD30B6dcF7C4e	Logi walle addr
signer	0x21cF8Ae13Bb72632562c6Fff438652Ba1a151bb0	Click Here
privateKey	0x4fd0a42218f3eae43a6ce26d22544e986139a01e5b34a62db53757ffca81bae1	Click Here

The nonce parameter is the current system time in microseconds. If it exceeds the system time or lags behind it by more than 10 seconds, the request is considered invalid.

```
#python
nonce = math.trunc(time.time()*1000000)
print(nonce)
#1748310859508867
```

```
#python
```

```
//java
```

```
Instant now = Instant.now();
```

```
long microsecond = now.getEpochSecond() * 1000000 + now.getNano() / 1000;
```

Example: Post an order (using Python as an example).

```

import time
import urllib
import threading

import requests
from eth_account.messages import encode_structured_data
from eth_account import Account

typed_data = {
    "types": {
        "EIP712Domain": [
            {"name": "name", "type": "string"},
            {"name": "version", "type": "string"},
            {"name": "chainId", "type": "uint256"},
            {"name": "verifyingContract", "type": "address"}
        ],
        "Message": [
            { "name": "msg", "type": "string" }
        ]
    },
    "primaryType": "Message",
    "domain": {
        "name": "AsterSignTransaction",
        "version": "1",
        "chainId": 1666,
        "verifyingContract": "0x0000000000000000000000000000000000000000"
    },
    "message": {
        "msg": "$msg"
    }
}

headers = {
    'Content-Type': 'application/x-www-form-urlencoded',
    'User-Agent': 'PythonApp/1.0'
}

host = 'https://sapi.asterdex.com'

# config your user and agent info here

```



```

user = '*'
signer = '*'
private_key = "*"

place_order = {"url":"/api/v3/order","method":"POST","params":{"symbol": "ASTERUSDT", "ty
                "timeInForce": "GTC", "quantity": "100", "price": "0.4"}}

_last_ms = 0
_i = 0
_nonce_lock = threading.Lock()

def get_nonce():
    global _last_ms, _i
    with _nonce_lock:
        now_ms = int(time.time())

        if now_ms == _last_ms:
            _i += 1
        else:
            _last_ms = now_ms
            _i = 0

        return now_ms * 1_000_000 + _i

def send_by_url(api) :
    my_dict = api['params']
    url = host + api['url']

    my_dict['nonce'] = str(get_nonce())
    my_dict['signer'] = signer

    param = urllib.parse.urlencode(my_dict)
    print(param)
    typed_data['message']['msg'] = param
    message = encode_structured_data(typed_data)
    signed = Account.sign_message(message, private_key=private_key)

    url = url + '?' + param + '&signature=' + signed.signature.hex()
    print(url)
    res = requests.post(url, headers=headers)

```

```

print(res.text)

def send_by_body(api) :
    my_dict = api['params']
    url = host +api['url']
    my_dict['nonce'] = str(get_nonce())
    my_dict['signer'] = signer

    param = urllib.parse.urlencode(my_dict)
    typed_data['message']['msg'] = param
    message = encode_structured_data(typed_data)

    signed = Account.sign_message(message, private_key=private_key)
    print(signed.signature.hex())

    my_dict['signature'] = signed.signature.hex()

    print(my_dict)
    res = requests.post(url, data=my_dict, headers=headers)

    print(res.text)

if __name__ == '__main__':
    send_by_url(place_order)
    # send_by_body(place_order)

```

Public API parameters

Terminology

The terminology in this section applies throughout the document. New users are encouraged to read it carefully for better understanding.

- **base asset** refers to the asset being traded in a trading pair, i.e., the asset name written first; for example, in **BTCUSDT**, **BTC** is the **base asset**.
- **quote asset** refers to the pricing asset of a trading pair, i.e., the asset name written at the latter part; for example, in **BTCUSDT**, **USDT** is the **quote asset**.

Enumeration definition

Trading pair status (status):

- TRADING - after trade

Trading pair type:

- PREDICTION - prediction

Order status (status):

Status	Description
NEW	Order accepted by the matching engine
PARTIALLY_FILLED	Part of the order was filled
FILLED	The order was fully filled
CANCELED	The user canceled the order
REJECTED	The order was not accepted by the matching engine and was not processed
EXPIRED	Order canceled by the trading engine, for example: Limit FOK order not filled, Market order not fully filled, orders canceled during exchange maintenance

Order types (orderTypes, type):

- LIMIT - Limit Order
- MARKET - Market Order

Order response type (newOrderRespType):

- ACK
- RESULT
- FULL

Order direction (direction side):

- BUY - Buy
- SELL - Sell

Valid types (timeInForce):

This defines how long an order can remain valid before expiring.

Status	Description
GTC (Good 'Til Canceled)	The order remains active until it is fully executed or manually canceled.
IOC (Immediate or Cancel)	The order will execute immediately for any amount available. Any unfilled portion is automatically canceled.
FOK (Fill or Kill)	The order must be fully executed immediately. If it cannot be filled in full, it is canceled right away.
GTX (Good till crossing, Post only)	The post-only limit order will only be placed if it can be added as a maker order and not as a taker order.

K-line interval:

m (minutes), h (hours), d (days), w (weeks), M (months)

- 1s (added)
- 1m
- 3m
- 5m
- 15m
- 30m
- 1h
- 2h
- 4h
- 6h
- 8h
- 12h
- 1d
- 3d

- 1w
- 1M

Rate limit type (rateLimitType)

REQUEST_WEIGHT

```
{
  "rateLimitType": "REQUEST_WEIGHT",
  "interval": "MINUTE",
  "intervalNum": 1,
  "limit": 1200
}
```

ORDERS

```
{
  "rateLimitType": "ORDERS",
  "interval": "MINUTE",
  "intervalNum": 1,
  "limit": 100
}
```

- REQUEST_WEIGHT - The maximum sum of request weights allowed within a unit time
- ORDERS - Order placement frequency limit per time unit

Interval restriction (interval)

- MINUTE - Minute

Filters

Filters, i.e. Filter, define a set of trading rules. There are two types: filters for trading pairs `symbol filters` , and filters for the entire exchange `exchange filters` (not supported yet)

Trading pair filters

PRICE_FILTER Price filter

Format in the /exchangeInfo response:

```
{
  "minPrice": "0.0001",
  "maxPrice": "0.9999",
  "filterType": "PRICE_FILTER",
  "tickSize": "0.01"
}
```

The `Price Filter` checks the validity of the `price` parameter in an order. It consists of the following three parts:

- `minPrice` defines the minimum allowed value for `price / stopPrice`.
- `maxPrice` defines the maximum allowed value for `price / stopPrice`.
- `tickSize` defines the step interval for `price / stopPrice`, meaning the price must equal `minPrice` plus an integer multiple of `tickSize`.

Each of the above items can be 0; when 0 it means that item is not constrained.

The logical pseudocode is as follows:

- `price >= minPrice`
- `price <= maxPrice`
- `(price - minPrice) % tickSize == 0`

PERCENT_PRICE price amplitude filter

Format in the /exchangeInfo response:

```
{
  "multiplierDown": "0.9500",
  "multiplierUp": "1.0500",
  "multiplierDecimal": "4",
  "filterType": "PERCENT_PRICE"
}
```

The `PERCENT_PRICE` filter defines the valid range of prices based on the index price.

For the "price percentage" to apply, the "price" must meet the following conditions:

- `price` $\leq$ `indexPrice` * `multiplierUp`
- `price` $\geq$ `indexPrice` * `multiplierDown`

LOT_SIZE order size

Format in the `/exchangeInfo` response:

```
{
  "stepSize": "0.00100000",
  "filterType": "LOT_SIZE",
  "maxQty": "100000.00000000",
  "minQty": "0.00100000"
}
```

Lots is an auction term. The `LOT_SIZE` filter validates the `quantity` (i.e., the amount) parameter in orders. It consists of three parts:

- `minQty` indicates the minimum allowed value for `quantity`.
- `maxQty` denotes the maximum allowed value for `quantity`.
- `stepSize` denotes the allowed step increment for `quantity`.

The logical pseudocode is as follows:

- `quantity` $\geq$ `minQty`
- `quantity` $\leq$ `maxQty`
- $(\text{quantity} - \text{minQty}) \% \text{stepSize} == 0$

MARKET_LOT_SIZE - Market order size

`*/exchangeInfo` response format:

```
{
  "stepSize": "0.00100000",
  "filterType": "MARKET_LOT_SIZE"
  "maxQty": "100000.00000000",
  "minQty": "0.00100000"
}
```

The `MARKET_LOT_SIZE` filter defines the `quantity` (i.e., the "lots" in an auction) rules for `MARKET` orders on a trading pair. There are three parts:

- `minQty` defines the minimum allowed `quantity`.
- `maxQty` defines the maximum allowed quantity.
- `stepSize` defines the increments by which the quantity can be increased or decreased.

In order to comply with the `market lot size`, the `quantity` must satisfy the following conditions:

- `quantity` $\geq$ `minQty`
- `quantity` $\leq$ `maxQty`
- $(\text{quantity} - \text{minQty}) \% \text{stepSize} == 0$

predictionEvents - Prediction Events

Format in the `*/exchangeInfo` response:


```

"predictionEvents": [
  {
    "eventName": "ETH_UP_DOWN_7D", // Prediction event name
    "oracleSymbol": "ETHUSDT", // Predicted symbol
    "predictionType": "UP_DOWN", // Prediction type
    "sides": [
      "UP",
      "DOWN"
    ], // Prediction directions
    "period": "10080M", // Prediction period in minutes
    "startTime": 1778476080000, // Event start time
    "endTime": 1780376880000, // Event end time
    "status": "LIVE", // Event status
    "currentRoundId": 3, // Current round ID
    "orderTypes": [
      "LIMIT",
      "MARKET"
    ], // Supported order types
    "timeInForce": [
      "GTC",
      "IOC",
      "FOK",
      "GTX"
    ], // Supported TIF values
    "rounds": [
      {
        "marketName": "USD1_UP_DOWN_5M_1778483280", // Prediction market name
        "symbols": [
          "USD1_UP_DOWN_5M_1778483280_YUSDT",
          "USD1_UP_DOWN_5M_1778483280_NUSDT"
        ], // Two symbols for the market; Y = up, N = down
        "assets": [
          "USD1_UP_DOWN_5M_1778483280_Y",
          "USD1_UP_DOWN_5M_1778483280_N"
        ], // Two assets for the market; Y = up, N = down
        "roundId": 2, // Round ID
        "tradingStartTime": 1778482970000, // Trading start time
        "tradingEndTime": 1778483520000, // Trading end time
        "windowStartTime": 1778483280000, // Window start time

```

```

        "windowEndTime": 1778483580000,      // Window end time
        "roundStatus": "SETTLED",           // Market status
        "openPrice": "0.99920000",         // Opening price
        "endPrice": "0.99910000",          // Ending price
        "winAsset": "USD1_UP_DOWM_5M_1778483280_N", // Winning asset: Y if u
        "settleTime": 1778483582057,       // Settlement time
        "settleHash": "0x3bc0aff52472dafd4b70d3423386a2dabf675fe7cd604e92391e"
    }
}
]

```

`status` is the event status:

Status	Description
LIVE	Running
CLOSED	Closed
INVALID	Invalid

`roundStatus` is the market status:

RoundStatus	Description
PREMARKET	Pre-market ready (trading available in advance)
TRADING	Market is open for trading
SETTLED	Market has been settled

- After settlement, the market will be automatically delisted after a period (24H). Delisted markets are no longer displayed.
- For events with `status` of `CLOSED` or `INVALID`, only `reduceOnly` Market Sell orders can be placed.

Market data API

Noop

Response:

```
{
  "code": 200,
  "msg": "success"
}
```

POST /api/v3/noop

By using this request, it is possible to efficiently cancel previously sent transactions that are still in the queue and have not completed the on chain operation (Nonce should be equal to this request 'no guarantee of success

)

Weight:

1

Test server connectivity

Response

```
{}
```

GET /api/v3/ping

Test whether the REST API can be reached.

Weight: 1

Parameters: NONE

Get server time

Response

```
{  
  "serverTime": 1499827319559  
}
```

GET /api/v3/time

Test if the REST API can be reached and retrieve the server time.

Weight: 1

Parameters: NONE

Trading specification information

Response

```
{
  "timezone": "UTC",
  "serverTime": 1756197279679,
  "rateLimits": [{
    "rateLimitType": "REQUEST_WEIGHT",
    "interval": "MINUTE",
    "intervalNum": 1,
    "limit": 6000
  },
  {
    "rateLimitType": "ORDERS",
    "interval": "MINUTE",
    "intervalNum": 1,
    "limit": 6000
  },
  {
    "rateLimitType": "ORDERS",
    "interval": "SECOND",
    "intervalNum": 10,
    "limit": 300
  }
],
  "exchangeFilters": [],
  "assets": [{
    "asset": "USD"
  }, {
    "asset": "USDT"
  },
  {
    "asset": "BNB"
  }
],
  "symbols": [{
    "status": "TRADING",
    "baseAsset": "BNB",
    "quoteAsset": "USDT",
    "pricePrecision": 8,
    "quantityPrecision": 8,
    "baseAssetPrecision": 8,
```

```
"quotePrecision": 8,
"filters": [{
    "minPrice": "0.0001",
    "maxPrice": "0.9999",
    "filterType": "PRICE_FILTER",
    "tickSize": "0.01000000"
},
{
    "stepSize": "0.00100000",
    "filterType": "LOT_SIZE",
    "maxQty": "1000",
    "minQty": "1"
},
{
    "stepSize": "0.00100000",
    "filterType": "MARKET_LOT_SIZE",
    "maxQty": "900000",
    "minQty": "0.00100000"
},
{
    "limit": 200,
    "filterType": "MAX_NUM_ORDERS"
},
{
    "minNotional": "5",
    "filterType": "MIN_NOTIONAL"
},
{
    "maxNotional": "100",
    "filterType": "MAX_NOTIONAL"
},
{
    "maxNotional": "100",
    "minNotional": "5",
    "avgPriceMins": 5,
    "applyMinToMarket": true,
    "filterType": "NOTIONAL",
    "applyMaxToMarket": true
},
{
```

```

        "multiplierDown": "0",
        "multiplierUp": "5",
        "multiplierDecimal": "0",
        "filterType": "PERCENT_PRICE"
    },
    {
        "bidMultiplierUp": "5",
        "askMultiplierUp": "5",
        "bidMultiplierDown": "0",
        "avgPriceMins": 5,
        "multiplierDecimal": "0",
        "filterType": "PERCENT_PRICE_BY_SIDE",
        "askMultiplierDown": "0"
    }
],
"orderTypes": [
    "LIMIT",
    "MARKET"
],
"timeInForce": [
    "GTC",
    "IOC",
    "FOK",
    "GTX"
],
"symbol": "BTC_UP_DOWN_5M_1778483280_YUSD",
"ocoAllowed": false
}]
}

```

GET /api/v3/exchangeInfo

Retrieve trading rules and trading pair information.

Weight: 1

Parameters: None

Depth information

Response

```
{
  "lastUpdateId": 1027024,
  "E":1589436922972, // Message output time
  "T":1589436922959, // Transaction time
  "bids": [
    [
      "4.00000000", // PRICE
      "431.00000000" // QTY
    ]
  ],
  "asks": [
    [
      "4.00000200",
      "12.00000000"
    ]
  ]
}
```

GET /api/v3/depth

Weight:

Based on limit adjustments:

Limitations	Weight
5, 10, 20, 50	2
100	5
500	10
1000	20

Parameters:

Name	Type	Is it required?	Description
symbol	STRING	YES	
limit	INT	NO	Default 100. Optional values: 5, 10, 20, 50, 100, 500, 1000

Recent trades list

Response

```
[
  {
    "id": 657,
    "price": "1.01000000",
    "qty": "5.00000000",
    "baseQty": "4.95049505",
    "time": 1755156533943,
    "isBuyerMaker": false
  }
]
```

GET /api/v3/trades

Get recent trades

Weight: 1

Parameters:

Name	Type	Is it required?	Description
symbol	STRING	YES	
limit	INT	NO	Default 500; maximum 1000

Query historical trades (MARKET_DATA)

Response

```
[
  {
    "id": 1140,
    "price": "1.10000000",
    "qty": "7.27200000",
    "baseQty": "6.61090909",
    "time": 1756094288700,
    "isBuyerMaker": false
  }
]
```

GET /api/v3/historicalTrades

Retrieve historical trades

Weight: 20

Parameters:

Name	Type	Is it required?	Description
symbol	STRING	YES	
limit	INT	NO	Default 500; maximum 1000.
fromId	LONG	NO	Return starting from which trade id. Defaults to returning the most recent trade records.

Recent trades (aggregated)

Response

```
[
  {
    "a": 26129, // Aggregate tradeId
    "p": "0.01633102", // Price
    "q": "4.70443515", // Quantity
    "f": 27781, // First tradeId
    "l": 27781, // Last tradeId
    "T": 1498793709153, // Timestamp
    "m": true, // Was the buyer the maker?
  }
]
```

GET /api/v3/aggTrades

The difference between aggregated trades and individual trades is that trades with the same price, same side, and same time are combined into a single entry.

Weight: 20

Parameters:

Name	Type	Is it required?	Description
symbol	STRING	YES	
fromId	LONG	NO	Return results starting from the trade ID that includes fromId
startTime	LONG	NO	Return results starting from trades after that time
endTime	LONG	NO	Return the trade records up to that moment
limit	INT	NO	Default 500; maximum 1000.

- If you send startTime and endTime, the interval must be less than one hour.
- If no filter parameters (fromId, startTime, endTime) are sent, the most recent trade records are returned by default

K-line data

Response

```
[
  [
    1499040000000, // Open time
    "0.01634790", // Open
    "0.80000000", // High
    "0.01575800", // Low
    "0.01577100", // Close
    "148976.11427815", // Volume
    1499644799999, // Close time
    "2434.19055334", // Quote asset volume
    308, // Number of trades
    "1756.87402397", // Taker buy base asset volume
    "28.46694368", // Taker buy quote asset volume
  ]
]
```

GET /api/v3/klines

Each K-line represents a trading pair. The open time of each K-line can be regarded as a unique ID.

Parameters:

Name	Type	Is it required?	Description
symbol	STRING	YES	
interval	ENUM	YES	See the enumeration definition: K-line interval
startTime	LONG	NO	
endTime	LONG	NO	
limit	INT	NO	Default 500; maximum 1500.

- If startTime and endTime are not sent, the most recent trades are returned by default

24h price change

Response

```
{
  "symbol": "BTC_UP_DOWNM_5M_1778483280_YUSD", //symbol
  "priceChange": "-94.99999800", //price change
  "priceChangePercent": "-95.960", //price change percent
  "weightedAvgPrice": "0.29628482", //weighted avgPrice
  "prevClosePrice": "3.89000000", //prev close price
  "lastPrice": "4.00000200", //last price
  "lastQty": "200.00000000", //last qty
  "bidPrice": "0.4999", //first bid price
  "bidQty": "431.00", //first bid qty
  "askPrice": "0.5001", //first ask price
  "askQty": "9.00", //first ask qty
  "openPrice": "99.00000000", //open price
  "highPrice": "100.00000000", //high price
  "lowPrice": "0.10000000", //low price
  "volume": "8913.30000000", //volume
  "quoteVolume": "15.30000000", //quote volume
  "openTime": 1499783499040, //open time
  "closeTime": 1499869899040, //close time
  "firstId": 28385, // first id
  "lastId": 28460, // last id
  "count": 76, // count
  "baseAsset": "BTC", //base asset
  "quoteAsset": "USD" //quote asset
}
```

GET /api/v3/ticker/24hr

24-hour rolling window price change data. Please note that omitting the symbol parameter will return data for all trading pairs; in that case the returned data is an example array for the respective pairs, which is not only large in volume but also has a very high weight.

Weight: 1 = single trading pair; **40** = When the trading pair parameter is missing (returns all trading pairs)

Parameters:

Name	Type	Is it required?	Description
symbol	STRING	NO	

- Please note that omitting the symbol parameter will return data for all trading pairs

Latest price

Response

```
{
  "symbol": "BTC_UP_DOWN_5M_1778483280_YUSD",
  "price": "1.30000000",
  "time": 164966690902
}
```

OR

```
[
  {
    "symbol": "BTC_UP_DOWN_5M_1778483280_YUSD",
    "price": "1.30000000",
    "time": 164966690902
  }
]
```

GET /api/v3/ticker/price

Get the latest price for a trading pair

Weight: 1 = Single trading pair; 2 = No symbol parameter (returns all pairs)

Parameters:

Name	Type	Is it required?	Description
symbol	STRING	NO	

- If no trading pair parameter is sent, information for all trading pairs will be returned

Current best order

Response

```
{
  "symbol": "BTC_UP_DOWN_5M_1778483280_YUSD",
  "bidPrice": "0.4999",
  "bidQty": "431.00",
  "askPrice": "0.5001",
  "askQty": "9.00"
  "time": 1589437530011 // Timestamp
}
```

OR

```
[
  {
    "symbol": "BTC_UP_DOWN_5M_1778483280_YUSD",
    "bidPrice": "0.4999",
    "bidQty": "431.00",
    "askPrice": "0.5001",
    "askQty": "9.00",
    "time": 1589437530011 // Timestamp
  }
]
```

GET /api/v3/ticker/bookTicker

Return the current best orders (highest bid, lowest ask)

Weight: 1 = Single trading pair; 2 = No symbol parameter (returns all pairs)

Parameters:

Name	Type	Is it required?	Description
symbol	STRING	NO	

- If no trading pair parameter is sent, information for all trading pairs will be returned

Get symbol fees

Response

```
{
  "symbol": "BTC_UP_DOWN_5M_1778483280_YUSD",
  "makerCommissionRate": "0.000200",
  "takerCommissionRate": "0.000700",
  "settleCommissionRate": "0.000100"
}
```

GET /api/v3/commissionRate

Get symbol fees

Weight: 20

Parameters:

Name	Type	Is it required?	Description
symbol	STRING	YES	

Spot account and trading API

Place order (TRADE)

Response ACK:


```
{
  "symbol": "BTC_UP_DOWN_5M_1778483280_YUSD",
  "orderId": 28,
  "clientOrderId": "6gCrw2kRUAf9CvJDGP16IP",
  "updateTime": 1507725176595,
  "price": "0.00000000",
  "avgPrice": "0.000000000000000000",
  "origQty": "10.00000000",
  "cumQty": "0",
  "executedQty": "10.00000000",
  "cumQuote": "10.00000000",
  "status": "FILLED",
  "timeInForce": "GTC",
  "stopPrice": "0",
  "origType": "LIMIT",
  "type": "LIMIT",
  "side": "SELL",
}
```

POST /api/v3/order

Send order

Weight: 1

Parameters:

Name	Type	Is it required?	Description
symbol	STRING	YES	
side	ENUM	YES	See enum definition: Order direction
type	ENUM	YES	See enumeration definition: Order type
timeInForce	ENUM	NO	See enum definition: Time in force
quantity	DECIMAL	NO	
quoteOrderQty	DECIMAL	NO	

Name	Type	Is it required?	Description
price	DECIMAL	NO	
newClientOrderId	STRING	NO	Client-customized unique order ID. If not provided, one will be generated automatically.
stopPrice	DECIMAL	NO	Only STOP, STOP_MARKET, TAKE_PROFIT, TAKE_PROFIT_MARKET require this parameter

Depending on the order `type` , certain parameters are mandatory:

Type	Mandatory parameters
LIMIT	timeInForce, quantity, price
MARKET	quantity or quoteOrderQty
STOP and TAKE_PROFIT	quantity, price, stopPrice
STOP_MARKET and TAKE_PROFIT_MARKET	quantity, stopPrice

Other information:

- Place a `MARKET SELL` market order; the user controls the amount of base assets to sell with the market order via `QUANTITY` .
 - For example, when placing a `MARKET SELL` market order on the `BTC_UP_DOWN_5M_1778483280_YUSD` pair, use `QUANTITY` to let the user specify how much BTC they want to sell.
- For a `MARKET BUY` market order, the user controls how much of the quote asset they want to spend with `quoteOrderQty` ; `QUANTITY` will be calculated by the system based on market liquidity. For example, when placing a `MARKET BUY` market order on the `BTC_UP_DOWN_5M_1778483280_YUSD` pair, use `quoteOrderQty` to let the user choose how much USD to use to buy BTC.
- A `MARKET` order using `quoteOrderQty` will not violate the `LOT_SIZE` limit rules; the order will be executed as closely as possible to the given `quoteOrderQty` .

- Unless a previous order has already been filled, orders set with the same `newClientOrderId` will be rejected.

Cancel order (TRADE)

Response

```
{
  "symbol": "BTC_UP_DOWNM_5M_1778483280_YUSDT",
  "orderId": 28,
  "clientOrderId": "6gCrw2kRUAf9CvJDGP16IP",
  "updateTime": 1507725176595,
  "price": "0.00000000",
  "avgPrice": "0.000000000000000000",
  "origQty": "10.00000000",
  "cumQty": "0",
  "executedQty": "10.00000000",
  "cumQuote": "10.00000000",
  "status": "CANCELED",
  "timeInForce": "GTC",
  "stopPrice": "0",
  "origType": "LIMIT",
  "type": "LIMIT",
  "side": "SELL",
}
```

DELETE /api/v3/order

Cancel active orders

Weight: 1

Parameters:

Name	Type	Is it required?	Description
symbol	STRING	YES	
orderId	LONG	NO	

Name	Type	Is it required?	Description
origClientOrderId	STRING	NO	

At least one of `orderId` or `origClientOrderId` must be sent.

Query order (USER_DATA)

Response

```
{
  "orderId": 38,
  "symbol": "BTC_UP_DOWN_5M_1778483280_YUSD",
  "status": "FILLED",
  "clientOrderId": "afMd4GBQyHkHpGWdiy34Li",
  "price": "20",
  "avgPrice": "12.000000000000000000",
  "origQty": "10",
  "executedQty": "10",
  "cumQuote": "120",
  "timeInForce": "GTC",
  "type": "LIMIT",
  "side": "BUY",
  "stopPrice": "0",
  "origType": "LIMIT",
  "time": 1649913186270,
  "updateTime": 1649913186297
}
```

GET `/api/v3/order`

Query order status

- Please note that orders meeting the following conditions will not be returned:
 - The final status of the order is `CANCELED` or `EXPIRED`, **and**
 - The order has no trade records, **and**
 - Order creation time + 7 days < current time

Weight: 1

Parameters:

Name	Type	Is it required?	Description
symbol	STRING	YES	
orderId	LONG	NO	
origClientOrderId	STRING	NO	

Note:

- You must send at least one of `orderId` or `origClientOrderId` .

Query Current Open Order (USER_DATA)

Response

```
{
  "orderId": 38,
  "symbol": "BTC_UP_DOWN_5M_1778483280_YUSD",
  "status": "NEW",
  "clientOrderId": "afMd4GBQyHkHpGWdiy34Li",
  "price": "20",
  "avgPrice": "12.0000000000000000",
  "origQty": "10",
  "executedQty": "10",
  "cumQuote": "120",
  "timeInForce": "GTC",
  "type": "LIMIT",
  "side": "BUY",
  "stopPrice": "0",
  "origType": "LIMIT",
  "time": 1649913186270,
  "updateTime": 1649913186297
}
```

GET /api/v3/openOrder

Query current open order status.

Weight: 1

Parameters:

Name	Type	Is it required?	Description
symbol	STRING	YES	
orderId	LONG	NO	
origClientOrderId	STRING	NO	

Note:

- You must send at least one of `orderId` or `origClientOrderId` .

Current open orders (USER_DATA)

Response

```
[
  {
    "orderId": 349661,
    "symbol": "BTC_UP_DOWN_5M_1778483280_YUSD",
    "status": "NEW",
    "clientOrderId": "LzypgiMwkf3TQ8wwvLo8RA",
    "price": "1.10000000",
    "avgPrice": "0.000000000000000000",
    "origQty": "5",
    "executedQty": "0",
    "cumQuote": "0",
    "timeInForce": "GTC",
    "type": "LIMIT",
    "side": "BUY",
    "stopPrice": "0",
    "origType": "LIMIT",
    "time": 1756252940207,
    "updateTime": 1756252940207,
  }
]
```

GET /api/v3/openOrders

Retrieve all current open orders for trading pairs. Use calls without a trading pair parameter with caution.

Weight:

- With symbol **1**
- Without **40**

Parameters:

Name	Type	Is it required?	Description
symbol	STRING	NO	

- If the symbol parameter is not provided, it will return the order books for all trading pairs.

Cancel All Open Orders (TRADE)

Response

```
{
  "code": 200,
  "msg": "The operation of cancel all open order is done."
}
```

DEL /api/v3/allOpenOrders

Weight:

• 1

Parameters:

Name	Type	Mandatory	Description
symbol	STRING	YES	
orderIdList	STRING	NO	orderid array string
origClientOrderIdList	STRING	NO	clientOrderId array string

Query all orders (USER_DATA)

Response


```
[
  {
    "orderId": 349661,
    "symbol": "BTC_UP_DOWN_5M_1778483280_YUSD",
    "status": "NEW",
    "clientOrderId": "LzypgiMwkf3TQ8wwvLo8RA",
    "price": "1.10000000",
    "avgPrice": "0.000000000000000000",
    "origQty": "5",
    "executedQty": "0",
    "cumQuote": "0",
    "timeInForce": "GTC",
    "type": "LIMIT",
    "side": "BUY",
    "stopPrice": "0",
    "origType": "LIMIT",
    "time": 1756252940207,
    "updateTime": 1756252940207,
  }
]
```

GET /api/v3/allOrders

Retrieve all account orders; active, canceled, or completed.

- Please note that orders meeting the following conditions will not be returned:
 - Order creation time + 7 days < current time

Weight: 5

Parameters:

Name	Type	Is it required?	Description
symbol	STRING	YES	
orderId	LONG	NO	
startTime	LONG	NO	

Name	Type	Is it required?	Description
endTime	LONG	NO	
limit	INT	NO	Default 500; maximum 1000

- The maximum query time range must not exceed 7 days.
- By default, query data is from the last 7 days.

GET /api/v3/transactionHistory

Response

```
[
  {
    "tranId": 1759115482304540227,
    "tradeId": null,
    "asset": "ASTER",
    "symbol": "BTC_UP_DOWN_5M_1778483280_YUSD",
    "balanceDelta": "-500.00000000",
    "balanceInfo": "TRADE_SOURCE",
    "time": 1759115482000,
    "type": "TRADE_SOURCE"
  }
]
```

Query transaction records

Weight:

30

Parameters:

Name	Type	Is it required?	Description
asset	STRING	NO	asset
type	STRING	NO	type
startTime	LONG	NO	startTime

Name	Type	Is it required?	Description
endTime	LONG	NO	endTime
limit	LONG	NO	default:100 max:1000

Note:

- type :
 TRADE_TARGET , TRADE_SOURCE , TRANSFER_SPOT_TO_FUTURE , TRANSFER_FUTURE_TO_SPOT ,
 TRANSFER_SPOT_TO_SPOT , AIRDROP , DIVIDEND , TRANSFER_REFUND , INTERNAL_TRANSFER ,
 TRANSFER , SWAP , COMMISSION_REBATE , CASH_BACK , STAKING_WITHDRAW ,
 STAKING_CLAIM , STAKING_DELEGATE
- If startTime and endTime are not provided, only data from the most recent 7 days will be returned.

Perp-spot transfer (TRADE)

Response:

```
{
  "tranId": 21841, //Tran Id
  "status": "SUCCESS" //Status
}
```

POST /api/v3/asset/wallet/transfer

Weight: 5

Parameters:

Name	Type	Is it required?	Description
amount	DECIMAL	YES	Quantity
asset	STRING	YES	Asset
clientTranId	STRING	YES	Transaction ID

Name	Type	Is it required?	Description
kindType	STRING	YES	Transaction type

- kindType FUTURE_SPOT(future to spot)/SPOT_FUTURE(spot to future)

Prediction market mint (TRADE)

Response

```
{
  "clientId": "xxx",           // Client order ID
  "quoteAmount": "10.00000000", // Total quote asset spent
  "yesPrice": "0.55000000",     // YES token price at mint time
  "noPrice": "0.45000000",     // NO token price at mint time (yesPrice + noPrice = 1
  "pairCount": "10.00000000"   // Number of YES+NO pairs minted
}
```

POST /api/v3/prediction/mint

Mint YES+NO token pairs for a prediction market. The system deducts an equivalent amount of quote asset at the current market price and credits equal quantities of YES and NO tokens to the account.

Weight: 1

Parameters:

Name	Type	Is it required?	Description
symbol	STRING	YES	The YES token trading pair of the prediction market
quantity	DECIMAL	YES	Number of YES+NO token pairs to mint
newClientId	STRING	NO	Client-customized unique order ID. If not provided, one will be generated

Name	Type	Is it required?	Description
			automatically.

Notes:

- `symbol` must be a valid prediction market token trading pair with status `TRADING` or `PENDING_TRADING`
- `quantity` represents the number of pairs to mint simultaneously; the account will receive equal amounts of YES and NO tokens
- `yesPrice` + `noPrice` = 1 (in quote asset units)

Prediction market burn (TRADE)

Response

```
{
  "clientOrderId": "xxx",           // Client order ID
  "quoteAmount": "9.80000000",     // Total quote asset returned after burn
  "yesPrice": null,                 // Not returned for burn operations
  "noPrice": null,                 // Not returned for burn operations
  "pairCount": "10.00000000"       // Number of YES+NO pairs burned
}
```

POST `/api/v3/prediction/burn`

Burn equal quantities of YES+NO token pairs to redeem the corresponding quote asset. The account must hold equal amounts of YES and NO tokens to perform a burn.

Weight: 1

Parameters:

Name	Type	Is it required?	Description
symbol	STRING	YES	The YES token trading pair of the

Name	Type	Is it required?	Description
			prediction market
quantity	DECIMAL	YES	Number of YES+NO token pairs to burn
newClientOrderId	STRING	NO	Client-customized unique order ID. If not provided, one will be generated automatically.

Notes:

- `symbol` must be a valid prediction market token trading pair with status `TRADING`, `PENDING_TRADING`, or `CLOSED`
- `quantity` represents the number of pairs to burn simultaneously; the account must hold the corresponding amounts of both YES and NO tokens
- `yesPrice` and `noPrice` are `null` in the burn response
- Burn operations are still available after the market is `CLOSED`

Query prediction market current positions (USER_DATA)

Response

```
[
  {
    "marketName": "BTC_UP_DOWN_20260512",          // Prediction market name
    "symbol": "BTC_UP_DOWN_5M_1778483280_YUSD",      // Trading pair
    "assetName": "USDT",                            // Settlement asset
    "type": "YES",                                   // Position direction: "YES" or "NO"
    "openAvgPrice": "0.65000000",                  // Average open price
    "cumQty": "100.00000000",                      // Total position quantity
    "closeAvgPrice": "0.00000000",                 // Average close price (0 if not closed)
    "realizedPnl": "0.00000000",                  // Realized PnL
    "closeQty": "0.00000000",                     // Closed quantity
    "insertTime": 1747000000000,                  // Position open time (millisecond timestamp)
    "commissionFee": "0.10000000",                // Commission fee paid
    "commissionAsset": "USDT",                    // Commission fee asset
    "balance": "65.00000000",                     // Position value (quantity × average open price)
    "availableBalance": "65.00000000",            // Available balance
  }
]
```

GET /api/v3/prediction/positions

Query the current account's prediction market position list.

Weight: 1

Parameters:

Name	Type	Is it required?	Description
symbol	STRING	NO	Trading pair; if not provided, returns all positions
signer	STRING	YES	API wallet address
nonce	STRING	YES	Current time in microseconds
signature	STRING	YES	Signature

Query prediction market position history (USER_DATA)

Response

```
[
  {
    "marketName": "BTC_UP_DOWN_20260501",           // Prediction market name
    "symbol": "BTC_UP_DOWNM_5M_1778483280_YUSD",    // Trading pair
    "assetName": "USDT",                           // Settlement asset
    "type": "YES",                                  // Position direction: "YES" or "NO"
    "openAvgPrice": "0.72000000",                  // Average open price
    "cumQty": "200.00000000",                      // Total position quantity
    "closeAvgPrice": "1.00000000",                  // Average close price
    "realizedPnl": "56.00000000",                   // Realized PnL
    "closeQty": "200.00000000",                     // Closed quantity
    "insertTime": 1746000000000,                    // Position open time (millisecond timestamp)
    "closeTime": 1746086400000,                     // Close/settlement time (millisecond timestamp)
    "status": "settled",                            // Position status: "close" (manually closed)
    "commissionFee": "0.10000000",                  // Commission fee paid
    "commissionAsset": "USDT",                      // Commission fee asset
    "settleFee": "2.00000000",                      // Settlement fee (only present when status is "close")
    "settleAsset": "USDT",                          // Settlement fee asset
  }
]
```

GET /api/v3/prediction/positionHistories

Query the current account's prediction market historical position list (closed or settled).

Weight: 1

Parameters:

Name	Type	Is it required?	Description
symbol	STRING	NO	Trading pair; if not provided, returns all historical

Name	Type	Is it required?	Description
			positions
startTime	LONG	NO	Start time (millisecond timestamp)
endTime	LONG	NO	End time (millisecond timestamp)
limit	INT	NO	Number of results; default 100, maximum 1000
signer	STRING	YES	API wallet address
nonce	STRING	YES	Current time in microseconds
signature	STRING	YES	Signature

Notes:

- When both `startTime` and `endTime` are provided, `startTime` must not be greater than `endTime`
- `limit` range is 1 to 1000, default is 100
- `status` field: `close` means the user manually closed the position; `settled` means the prediction market expired and settled

Query prediction market settlement histories (USER_DATA)

Response

```
[
  {
    "marketName": "BTC_UP_DOWN_20260501", // Prediction market name
    "asset": "BTC_UP_DOWN_20260501YES", // Token asset (YES or NO token)
    "symbol": "BTC_UP_DOWN_20260501YES", // Trading pair
    "marketStartTime": 1746000000000, // Market start time (millisecond timestamp)
    "marketEndTime": 1746086400000, // Market end time / settlement time (millisecond timestamp)
    "shareAmount": "200.00000000", // Share quantity held at settlement
    "settleAsset": "USDT", // Settlement asset
    "settleAmount": "200.00000000", // Settlement amount received
    "settleFeeAmount": "2.00000000", // Settlement fee charged
    "entryPrice": "0.72000000", // Average entry price
    "settlePrice": "1.00000000", // Final settlement price
    "realizedPnl": "56.00000000", // Realized PnL after settlement
    "status": "settled" // Settlement status
  }
]
```

GET /api/v3/prediction/settlementHistories

Query the current account's prediction market settlement history (markets that have expired and settled).

Weight: 1

Parameters:

Name	Type	Is it required?	Description
symbol	STRING	NO	Trading pair; if not provided, returns all settlement records
startTime	LONG	NO	Filter by market start time (millisecond timestamp)
endTime	LONG	NO	Filter by market end time (millisecond timestamp)
limit	INT	NO	Number of results; default 100, maximum 1000

Name	Type	Is it required?	Description
signer	STRING	YES	API wallet address
nonce	STRING	YES	Current time in microseconds
signature	STRING	YES	Signature

Notes:

- When both `startTime` and `endTime` are provided, `startTime` must not be greater than `endTime`
- `limit` range is 1 to 1000, default is 100
- `settleAmount` is the gross payout; `realizedPnl` equals `settleAmount` minus the cost basis
- `settleFeeAmount` is deducted from the settlement payout

Get withdraw fee (NONE)

Response:

```
{
  "tokenPrice": 1.00019000,
  "gasCost": 0.5000,
  "gasUsdValue": 0.5
}
```

GET `/api/v3/aster/withdraw/estimateFee`

Weight:

1

Parameters:

Name	Type	Mandatory	Description
chainId	STRING	YES	

Name	Type	Mandatory	Description
asset	STRING	YES	

Notes:

- chainId: 1(ETH),56(BSC),42161(Arbi)
- gasCost: The minimum fee required for a withdrawal

Withdraw (USER_DATA)

Response:

```
{
  "withdrawId": "1014729574755487744",
  "hash": "0xa6d1e617a3f69211df276fdd8097ac8f12b6ad9c7a49ba75bbb24f002df0ebb"
}
```

POST /api/v3/aster/user-withdraw

Weight:

5

Parameters:

Name	Type	Mandatory	Description
chainId	STRING	YES	1(ETH),56(BSC),42161(Arbi)
asset	STRING	YES	
amount	STRING	YES	
fee	STRING	YES	
receiver	STRING	YES	The address of the current account
nonce	STRING	YES	The current time in microseconds
userSignature	STRING	YES	

Note:

- chainId: 1(ETH),56(BSC),42161(Arbi)
- receiver: The address of the current account
- If the futures account balance is insufficient, funds will be transferred from the spot account to the perp account for withdrawal.
- userSignature demo

```

const domain = {
  name: 'Aster',
  version: '1',
  chainId: 56,
  verifyingContract: ethers.ZeroAddress,
}

const currentTime = Date.now() * 1000

const types = {
  Action: [
    {name: "type", type: "string"},
    {name: "destination", type: "address"},
    {name: "destination Chain", type: "string"},
    {name: "token", type: "string"},
    {name: "amount", type: "string"},
    {name: "fee", type: "string"},
    {name: "nonce", type: "uint256"},
    {name: "aster chain", type: "string"},
  ],
}

const value = {
  'type': 'Withdraw',
  'destination': '0xD9cA6952F1b1349d27f91E4fa6FB8ef67b89F02d',
  'destination Chain': 'BSC',
  'token': 'USDT',
  'amount': '10.123400',
  'fee': '1.234567891',
  'nonce': currentTime,
  'aster chain': 'Mainnet',
}

const signature = await signer.signTypedData(domain, types, value)

```

Account information (USER_DATA)

Response

```
{
  "feeTier": 0,
  "canTrade": true,
  "canDeposit": true,
  "canWithdraw": true,
  "canBurnAsset": true,
  "updateTime": 0,
  "balances": [
    {
      "asset": "BTC",
      "free": "4723846.89208129",
      "locked": "0.00000000"
    },
    {
      "asset": "LTC",
      "free": "4763368.68006011",
      "locked": "0.00000000"
    }
  ]
}
```

GET /api/v3/account

Retrieve current account information

Weight: 5

Parameters:

Name	Type	Is it required?	Description
------	------	-----------------	-------------

Account trade history (USER_DATA)

Response

```
[
  {
    "symbol": "BTC_UP_DOWN_5M_1778483280_YUSD",
    "id": 1002,
    "orderId": 266358,
    "side": "BUY",
    "price": "1",
    "qty": "2",
    "quoteQty": "2",
    "commission": "0.00105000",
    "commissionAsset": "BNB",
    "time": 1755656788798,
    "counterpartyId": 19,
    "createUpdateId": null,
    "maker": false,
    "buyer": true
  }
]
```

GET /api/v3/userTrades

Retrieve the trade history for a specified trading pair of an account

Weight: 5

Parameters:

Name	Type	Is it required?	Description
symbol	STRING	NO	
orderId	LONG	NO	Must be used together with the parameter symbol
startTime	LONG	NO	
endTime	LONG	NO	
fromId	LONG	NO	Starting trade ID. Defaults to fetching the most

Name	Type	Is it required?	Description
			recent trade.
limit	INT	NO	Default 500; maximum 1000

- If both `startTime` and `endTime` are not sent, only data from the last 7 days will be returned.
- The maximum interval between `startTime` and `endTime` is 7 days.
- `fromId` cannot be sent together with `startTime` or `endTime`.

WebSocket market data feed

- The base URL for all wss endpoints listed in this document is:
wss://sstream.asterdex.com
- Streams have either a single raw stream or a combined stream
- Single raw streams format is `**/ws/**`
- The URL format for combined streams is `**/stream?streams=/**`
- When subscribing to combined streams, the event payload is wrapped in this format:
`**{"stream":"","data":}**`
- All trading pairs in stream names are **lowercase**
- Each link to sstream.asterdex.com is valid for no more than 24 hours; please handle reconnections appropriately
- Every 3 minutes the server sends a ping frame; the client must reply with a pong frame within 10 minutes, otherwise the server will close the connection. The client is allowed to send unpaired pong frames (i.e., the client may send pong frames at a frequency higher than once every 10 minutes to keep the connection alive).

Real-time subscribe/unsubscribe data streams

- The following messages can be sent via WebSocket to subscribe or unsubscribe to data streams. Examples are shown below.

- The `id` in the response content is an unsigned integer that serves as the unique identifier for exchanges of information.
- If the `result` in the response content is `null`, it indicates the request was sent successfully.

Subscribe to a stream

Response

```
{
  "result": null,
  "id": 1
}
```

- **Request** { "method": "SUBSCRIBE", "params":
"btc_updownm5m1778483280_usdt@aggTrade", "btc_updownm5m1778483280_usdt@depth"
 , "id": 1 }

Unsubscribe from a stream

Response

```
{
  "result": null,
  "id": 312
}
```

- **Request** { "method": "UNSUBSCRIBE", "params":
"btc_updownm5m1778483280_usdt@depth"
 , "id": 312 }

Subscribed to the feed

Response

```
{
  "result": [
    "btc_up_down_5m_1778483280_yusdt@aggTrade"
  ],
  "id": 3
}
```

- **Request**

```
{ "method": "LIST_SUBSCRIPTIONS", "id": 3 }
```

Set properties

Currently, the only configurable property is whether to enable the `combined` ("combined") stream. When connecting using `/ws/` ("raw stream"), the combined property is set to `false`, while connecting using `/stream/` sets the property to `true`.

Response

```
{
  "result": null,
  "id": 5
}
```

- **Request** { "method": "SET_PROPERTY" "params":

"combined", true

```
, "id": 5 }
```

Retrieve properties

Response

```
{
  "result": true, // Indicates that combined is set to true.
  "id": 2
}
```

- **Request**

```
{ "method": "GET_PROPERTY", "params":
```

”combined”

```
, "id": 2 }
```

Error message

Error message	Description
<code>{"code": 0, "msg": "Unknown property"}</code>	Parameters applied in SET_PROPERTY or GET_PROPERTY are invalid
<code>{"code": 1, "msg": "Invalid value type: expected Boolean", "id": '%s'}</code>	Only true or false are accepted
<code>{"code": 2, "msg": "Invalid request: property name must be a string"}</code>	The provided attribute name is invalid
<code>{"code": 2, "msg": "Invalid request: request ID must be an unsigned integer"}</code>	Parameter ID not provided or ID has an invalid type
<code>{"code": 2, "msg": "Invalid request: unknown variant %s, expected one of SUBSCRIBE, UNSUBSCRIBE, LIST_SUBSCRIPTIONS, SET_PROPERTY, GET_PROPERTY at line 1 column 28"}</code>	Typo warning, or the provided value is not of the expected type
<code>{"code": 2, "msg": "Invalid request: too many parameters"}</code>	Unnecessary parameters were provided in the data
<code>{"code": 2, "msg": "Invalid request: property name must be a string"}</code>	Property name not provided
<code>{"code": 2, "msg": "Invalid request: missing field method at line 1 column 73"}</code>	Data did not provide method

Error message	Description
<code>{"code":3,"msg":"Invalid JSON: expected value at line %s column %s"}</code>	JSON syntax error

Collection transaction flow

Payload:

```
{
  "e": "aggTrade", // Event type
  "E": 123456789,  // Event time
  "s": "BTC_UP_DOWN_5M_1778483280_YUSD", // Symbol
  "a": 12345,      // Aggregate trade ID
  "p": "0.001",    // Price
  "q": "100",      // Quantity
  "f": 100,        // First trade ID
  "l": 105,        // Last trade ID
  "T": 123456785,  // Trade time
  "m": true,       // Is the buyer the market maker?
  "M": true        // Ignore
}
```

The collection transaction stream pushes transaction information and is an aggregation of a single order.

Stream name: `<symbol>@aggTrade`

Update speed: real-time

Tick-by-tick trades

Payload:

```
{
  "e": "trade",      // Event type
  "E": 123456789,    // Event time
  "s": "BTC_UP_DOWN_5M_1778483280_YUSD", // Symbol
  "t": 12345,        // Trade ID
  "p": "0.001",      // Price
  "q": "100",        // Quantity
  "T": 123456785,    // Trade time
  "m": true,         // Is the buyer the market maker?
}
```

Stream name: `<symbol>@trade`

Each trade stream pushes the details of every individual trade. A **trade**, also called a transaction, is defined as a match between exactly one taker and one maker.

K-line streams

Payload:

```

{
  "e": "kline",      // Event type
  "E": 123456789,    // Event time
  "s": "BTC_UP_DOWN_5M_1778483280_YUSD", // Symbol
  "k": {
    "t": 123400000, // Kline start time
    "T": 123460000, // Kline close time
    "s": "BTC_UP_DOWN_5M_1778483280_YUSD", // Symbol
    "i": "1m",      // Interval
    "f": 100,        // First trade ID
    "L": 200,        // Last trade ID
    "o": "0.0010",   // Open price
    "c": "0.0020",   // Close price
    "h": "0.0025",   // High price
    "l": "0.0015",   // Low price
    "v": "1000",     // Base asset volume
    "n": 100,        // Number of trades
    "x": false,      // Is this kline closed?
    "q": "1.0000",   // Quote asset volume
    "V": "500",      // Taker buy base asset volume
    "Q": "0.500",    // Taker buy quote asset volume
    "B": "123456"    // Ignore
  }
}

```

The K-line stream pushes per-second updates for the requested type of K-line (the latest candle).

Stream name: `<symbol>@kline_<interval>`

Update speed: 2000ms

K-line interval parameter:

m (minutes), h (hours), d (days), w (weeks), M (months)

- 1m
- 3m
- 5m

- 15m
- 30m
- 1h
- 2h
- 4h
- 6h
- 8h
- 12h
- 1d
- 3d
- 1w
- 1M

Simplified ticker by symbol

Payload:

```
{
  "e": "24hrMiniTicker", // Event type
  "E": 123456789,        // Event time
  "s": "BTC_UP_DOWN_5M_1778483280_YUSD", // Symbol
  "c": "0.0025",         // Close price
  "o": "0.0010",         // Open price
  "h": "0.0025",         // High price
  "l": "0.0010",         // Low price
  "v": "10000",          // Total traded base asset volume
  "q": "18"              // Total traded quote asset volume
}
```

Refreshed simplified 24-hour ticker information by symbol

Stream name: <symbol>@miniTicker

Update speed: 1000ms

Compact tickers for all symbols in the entire market

Payload:

```
[
  {
    // Same as <symbol>@miniTicker payload
  }
]
```

Same as above, but pushes all trading pairs. Note that only updated tickers will be pushed.

Stream name: !miniTicker@arr

Update speed: 1000ms

Full ticker per symbol

Payload:

```
{
  "e": "24hrTicker", // Event type
  "E": 123456789,     // Event time
  "s": "BTC_UP_DOWN_5M_1778483280_YUSD", // Symbol
  "p": "0.0015",     // Price change
  "P": "250.00",     // Price change percent
  "w": "0.0018",     // Weighted average price
  "c": "0.0025",     // Last price
  "Q": "10",         // Last quantity
  "o": "0.0010",     // Open price
  "h": "0.0025",     // High price
  "l": "0.0010",     // Low price
  "v": "10000",      // Total traded base asset volume
  "q": "18",         // Total traded quote asset volume
  "O": 0,            // Statistics open time
  "C": 86400000,     // Statistics close time
  "F": 0,            // First trade ID
  "L": 18150,        // Last trade Id
  "n": 18151         // Total number of trades
}
```

Pushes per-second tag statistics for a single trading pair over a rolling 24-hour window.

Stream name: <symbol>@ticker

Update speed: 1000ms

Complete ticker for all trading pairs on the entire market

Payload:

```
[
  {
    // Same as <symbol>@ticker payload
  }
]
```

Pushes the full 24-hour refreshed ticker information for all trading pairs across the entire market. Note that tickers without updates will not be pushed.

Stream name: `!ticker@arr`

Update speed: 1000ms

Best order book information by symbol

Payload:

```
{
  "u": 400900217,      // order book updateId
  "s": "BTC_UP_DOWN_5M_1778483280_YUSD", // symbol
  "b": "25.35190000", // best bid price
  "B": "31.21000000", // best bid qty
  "a": "25.36520000", // best ask price
  "A": "40.66000000"  // best ask qty
}
```

Real-time push of best order book information for the specified trading pair

Stream name: `<symbol>@bookTicker`

Update speed: Real-time

Best order book information across the entire market

Payload:

```
{
  // Same as <symbol>@bookTicker payload
}
```

Real-time push of the best order information for all trading pairs

Stream name: !bookTicker

Update speed: Real-time

Limited depth information

Payload:

```
{
  "e": "depthUpdate", // Event type
  "E": 123456789,     // Event time
  "T": 123456788,     // Transaction time
  "s": "BTC_UP_DOWN_5M_1778483280_YUSD", // Symbol
  "U": 100,           // First update ID in event
  "u": 120,           // Final update ID in event
  "pu": 99,           // Final update Id in last stream(ie `u` in last stream)
  "bids": [           // Bids to be updated
    [
      "0.0024",       // Price level to be updated
      "10"             // Quantity
    ]
  ],
  "asks": [           // Asks to be updated
    [
      "0.0026",       // Price level to be updated
      "100"            // Quantity
    ]
  ]
}
```

Limited depth information pushed every second or every 100 milliseconds. Levels indicate how many levels of bid/ask information, optional 5/10/20 levels.

Stream names: <symbol>@depth<levels> or <symbol>@depth<levels>@100ms .

Update speed: 1000ms or 100ms

Incremental depth information

Payload:

```
{
  "e": "depthUpdate", // Event type
  "E": 123456789,     // Event time
  "T": 123456788,     // Transaction time
  "s": "BTC_UP_DOWN_5M_1778483280_YUSD", // Symbol
  "U": 100,           // First update ID in event
  "u": 120,           // Final update ID in event
  "pu": 99,           // Final update Id in last stream(ie `u` in last stream)
  "b": [              // Bids to be updated
    [
      "5.4",          // Price level to be updated
      "10"             // Quantity
    ]
  ],
  "a": [              // Asks to be updated
    [
      "5.6",          // Price level to be updated
      "100"            // Quantity
    ]
  ]
}
```

Pushes the changed parts of the orderbook (if any) every second or every 100 milliseconds

Stream name: <symbol>@depth or <symbol>@depth@100ms

Update speed: 1000ms or 100ms

How to correctly maintain a local copy of an order book

1. Subscribe to **wss://sstream.asterdex.com/ws/btc_up_down_5m_1778483280_yusdt@depth**

2. Start caching the received updates. For the same price level, later updates overwrite earlier ones.
3. Fetch the REST endpoint **https://sapi.asterdex.com/api/v3/depth?symbol=BTC_UP_DOWM_5M_1778483280_YUSDT&limit=1000** to obtain a 1000-level depth snapshot
4. Discard from the currently cached messages those with `u` $\leq$ the `lastUpdateId` obtained in step 3 (drop older, expired information)
5. Apply the depth snapshot to your local order book copy, and resume updating the local copy from the first WebSocket event whose `U` $\leq$ `lastUpdateId + 1` **and** `u` $\geq$ `lastUpdateId + 1`
6. Each new event's `U` should equal exactly the previous event's `u + 1`; otherwise packets may have been lost - restart initialization from step 3
7. The order quantity in each event represents the current order quantity at that price as an **absolute value**, not a relative change
8. If the order quantity at a given price is 0, it means the orders at that price have been canceled or filled, and that price level should be removed

WebSocket account information push

- The base URL for the API endpoints listed in this document is:
<https://sapi.asterdex.com>
- The `listenKey` used to subscribe to account data is valid for 60 minutes from the time of creation
- You can extend the 60-minute validity of a `listenKey` by sending a `PUT` request
- You can immediately close the current data stream and invalidate the `listenKey` by sending a `DELETE` for a `listenKey`
- Sending a `POST` on an account with a valid `listenKey` will return the currently valid `listenKey` and extend its validity by 60 minutes
- The WebSocket interface baseurl: **<wss://sstream.asterdex.com>**
- The stream name for subscribing to the user account data stream is `**/ws/**`
- Each connection is valid for no more than 24 hours; please handle disconnections and reconnections appropriately

Listen Key (spot account)

Generate Listen Key (USER_STREAM)

Response

```
{
  "listenKey": "pqia91ma19a5s61cv6a81va65sdf19v8a65a1a5s61cv6a81va65sdf19v8a65a1"
}
```

POST /api/v3/listenKey

Start a new data stream. The data stream will be closed after 60 minutes unless a keepalive is sent. If the account already has a valid `listenKey` , that `listenKey` will be returned and its validity extended by 60 minutes.

Weight: 1

Parameters: NONE

Extend Listen Key validity period (USER_STREAM)

Response

```
{}
```

PUT /api/v3/listenKey

Validity extended to 60 minutes after this call. It is recommended to send a ping every 30 minutes.

Weight: 1

Parameters:

Name	Type	Is it required?	Description
listenKey	STRING	YES	

Close Listen Key (USER_STREAM)

Response

```
{}
```

```
DELETE /api/v3/listenKey
```

Close user data stream

Weight: 1

Parameters:

Name	Type	Is it required?	Description
listenKey	STRING	YES	

Payload: ACCOUNT_UPDATE

An `outboundAccountPosition` event is sent whenever an account balance changes; it contains the assets that may have changed due to the event that generated the balance update.

Payload


```
{
  "B": [ //Balance
    {
      "a": "SLP25", //Asset
      "f": "10282.42029415", //Free
      "l": "653.00000001" //Locked
    },
    {
      "a": "ADA25",
      "f": "9916.96229880",
      "l": "34.00510000"
    }
  ],
  "e": "outboundAccountPosition", //Event type
  "T": 1649926447190, //Time of last account update
  "E": 1649926447205 //Event Time
  "m": "WITHDRAW" // Event reason type
}
```

Payload: Order Update

Orders are updated via the `executionReport` event

Payload

```

{
  "s": "BTC_UP_DOWN_5M_1778483280_YUSD", // symbol
  "c": "Xzh0gnxT41PStbwq0tXnjD", // client order id
  "S": "SELL", // order direction
  "o": "LIMIT", // order type
  "f": "GTC", // Time in force
  "q": "10.001000", // Order quantity
  "p": "19.1000000000", // Order price
  "ap": "19.099999995550656", //average price
  "P": "0", //stop price
  "x": "TRADE", // Current execution type
  "X": "PARTIALLY_FILLED", // Current order status
  "i": 27, // Order ID
  "l": "1", // Last executed quantity
  "z": "8.999000", // Cumulative filled quantity
  "L": "19.1000000000", // Last executed price
  "n": "0.00382000", // Commission amount
  "N": "SLP25", // Commission asset
  "T": 1649926447190, //Transaction Time
  "t": 18, // transaction id
  "m": true, // is this trade the maker side?
  "ot": "LIMIT", //original order type
  "O": 0, // Order creation time
  "Z": "171.88089996", // Cumulative quote asset transacted quantity
  "Y": "19.1000000000000000", // Last quote asset transacted quantity (i.e. lastPrice *
  "Q": "0", // Quote Order Qty
  "e": "executionReport", // event
  "E": 1649926447209 // event time
}

```

Execution type:

- NEW - New Order
- CANCELED - Order canceled
- REJECTED - New order was rejected
- TRADE - Order had a new fill
- EXPIRED - Order expired (based on the order's Time In Force parameter)

Event: TradePro

Topic Subscribe:

```
{
  "method": "SUBSCRIBE",
  "params": [
    "btc_up_down_5m_1778483280_yusdt@tradepro"
  ],
  "id": 3
}
```

Payload:

```
{
  "stream": "btc_up_down_5m_1778483280_yusdt@tradepro",
  "data": {
    "e": "tradepro",
    "E": 1773751963081,
    "T": 1773751963079,
    "s": "BTC_UP_DOWN_5M_1778483280_YUSD",
    "t": 128884613,
    "p": "73685.5",
    "q": "0.297",
    "h": "0X0000000000000000000000000000000000000000000000000000000000000000",
    "m": [
      "hidden",
      "hidden"
    ]
  }
}
```

- h: Transaction hash of the trade.
- m: Array containing the participant addresses:
 - m[0]: Taker address
 - m[1]: Maker address

#错误代码

error JSON payload:

```
{  
  "code":-1121,  
  "msg":"Invalid symbol."  
}
```

Errors consist of two parts: an error code and a message. The code is standardized, but the message may vary.

10xx - General server or network issues

-1000 UNKNOWN

- An unknown error occurred while processing the request.

-1001 DISCONNECTED

- Internal error; unable to process your request. Please try again.

-1002 UNAUTHORIZED

- You are not authorized to execute this request.

-1003 TOO_MANY_REQUESTS

- Too many requests queued.
- Too many requests; please use the WebSocket for live updates.
- Too many requests; current limit is %s requests per minute. Please use the WebSocket for live updates to avoid polling the API.
- Too many request weights; IP banned until %s. Please use the WebSocket for live updates to avoid bans.

-1004 DUPLICATE_IP

- This IP is already on the white list.

-1005 NO_SUCH_IP

- No such IP has been whitelisted.

-1006 UNEXPECTED_RESP

- An unexpected response was received from the message bus. Execution status unknown.

-1007 TIMEOUT

- Timeout waiting for response from backend server. Send status unknown; execution status unknown.

-1014 UNKNOWN_ORDER_COMPOSITION

- The current order parameter combination is not supported.

-1015 TOO_MANY_ORDERS

- Too many new orders.
- Too many new orders; the current limit is %s orders per %s.

-1016 SERVICE_SHUTTING_DOWN

- This service is no longer available.

-1020 UNSUPPORTED_OPERATION

- This operation is not supported.

-1022 INVALID_SIGNATURE

- The signature for this request is invalid.

-1023 START_TIME_GREATER_THAN_END_TIME

- The start time in the parameters is after the end time.

11xx - Request issues

-1100 ILLEGAL_CHARS

- Illegal characters found in a parameter.
- Illegal characters found in parameter %s; legal range is %s.

-1101 TOO_MANY_PARAMETERS

- Too many parameters sent for this endpoint.
- Too many parameters; expected %s and received %s.
- Duplicate values for a parameter detected.

-1102 MANDATORY_PARAM_EMPTY_OR_MALFORMED

- A mandatory parameter was not sent, was empty/null, or malformed.
- Mandatory parameter %s was not sent, was empty/null, or malformed.
- Param %s or %s must be sent, but both were empty/null.

-1103 UNKNOWN_PARAM

- An unknown parameter was sent.

-1104 UNREAD_PARAMETERS

- Not all sent parameters were read.
- Not all sent parameters were read; read %s parameter(s) but %s parameter(s) were sent.

-1105 PARAM_EMPTY

- A parameter was empty.
- Parameter %s was empty.

-1106 PARAM_NOT_REQUIRED

- A parameter was sent when not required.

-1111 BAD_PRECISION

- The precision exceeds the maximum defined for this asset.

-1112 NO_DEPTH

- No open orders for the trading pair.

-1114 TIF_NOT_REQUIRED

- TimeInForce parameter sent when not required.

-1115 INVALID_TIF

- Invalid timeInForce.

-1116 INVALID_ORDER_TYPE

- Invalid orderType.

-1117 INVALID_SIDE

- Invalid order side.

-1118 EMPTY_NEW_CL_ORD_ID

- New client order ID was empty.

-1119 EMPTY_ORG_CL_ORD_ID

- The client's custom order ID is empty.

-1120 BAD_INTERVAL

- Invalid time interval.

-1121 BAD_SYMBOL

- Invalid trading pair.

-1125 INVALID_LISTEN_KEY

- This listenKey does not exist.

-1127 MORE_THAN_XX_HOURS

- The query interval is too large.
- More than %s hours between startTime and endTime.

-1128 OPTIONAL_PARAMS_BAD_COMBO

- Combination of optional parameters invalid.

-1130 INVALID_PARAMETER

- The parameter sent contains invalid data.
- Data sent for parameter %s is not valid.

-1136 INVALID_NEW_ORDER_RESP_TYPE

- Invalid newOrderRespType.

20xx - Processing Issues

-2010 NEW_ORDER_REJECTED

- New order rejected.

-2011 CANCEL_REJECTED

- Order cancellation rejected.

-2013 NO_SUCH_ORDER

- Order does not exist.

-2014 BAD_API_KEY_FMT

- API-key format invalid.

-2015 REJECTED_MBX_KEY

- Invalid API key, IP, or permissions for action.

-2016 NO_TRADING_WINDOW

- No trading window could be found for the symbol. Try ticker/24hrs instead.

-2018 BALANCE_NOT_SUFFICIENT

- Balance is insufficient.

-2020 UNABLE_TO_FILL

- Unable to fill.

-2021 ORDER_WOULD_IMMEDIATELY_TRIGGER

- Order would immediately trigger.

-2022 REDUCE_ONLY_REJECT

- ReduceOnly Order is rejected.

-2024 POSITION_NOT_SUFFICIENT

- Position is not sufficient.

-2025 MAX_OPEN_ORDER_EXCEEDED

- Reached max open order limit.

-2026 REDUCE_ONLY_ORDER_TYPE_NOT_SUPPORTED

- This OrderType is not supported when reduceOnly.

40xx - Filters and other Issues

-4000 INVALID_ORDER_STATUS

- Invalid order status.

-4001 PRICE_LESS_THAN_ZERO

- Price less than 0.

-4002 PRICE_GREATER_THAN_MAX_PRICE

- Price greater than max price.

-4003 QTY_LESS_THAN_ZERO

- Quantity less than zero.

-4004 QTY_LESS_THAN_MIN_QTY

- Quantity less than minimum quantity.

-4005 QTY_GREATER_THAN_MAX_QTY

- Quantity greater than maximum quantity.

-4006 STOP_PRICE_LESS_THAN_ZERO

- Stop price less than zero.

-4007 STOP_PRICE_GREATER_THAN_MAX_PRICE

- Stop price greater than max price.

-4008 TICK_SIZE_LESS_THAN_ZERO

- Tick size less than zero.

-4009 MAX_PRICE_LESS_THAN_MIN_PRICE

- Max price less than min price.

-4010 MAX_QTY_LESS_THAN_MIN_QTY

- Maximum quantity less than minimum quantity.

-4011 STEP_SIZE_LESS_THAN_ZERO

- Step size less than zero.

-4012 MAX_NUM_ORDERS_LESS_THAN_ZERO

- Maximum order quantity less than 0.

-4013 PRICE_LESS_THAN_MIN_PRICE

- Price less than minimum price.

-4014 PRICE_NOT_INCREASED_BY_TICK_SIZE

- Price not increased by tick size.

-4015 INVALID_CL_ORD_ID_LEN

- Client order ID is not valid.
- Client order ID length should not be more than 36 characters.

-4016 PRICE_HIGHTER_THAN_MULTIPLIER_UP

- Price is higher than mark price multiplier cap.

-4017 MULTIPLIER_UP_LESS_THAN_ZERO

- Multiplier up less than zero.

-4018 MULTIPLIER_DOWN_LESS_THAN_ZERO

- Multiplier down less than zero.

-4019 COMPOSITE_SCALE_OVERFLOW

- Composite scale too large.

-4020 TARGET_STRATEGY_INVALID

- Target strategy invalid for orderType %s, reduceOnly %b'

-4021 INVALID_DEPTH_LIMIT

- Invalid depth limit.
- %s is not a valid depth limit.

-4022 WRONG_MARKET_STATUS

- Market status sent is not valid.

-4023 QTY_NOT_INCREASED_BY_STEP_SIZE

- The increment of the quantity is not a multiple of the step size.

-4024 PRICE_LOWER_THAN_MULTIPLIER_DOWN

- Price is lower than mark price multiplier floor.

-4025 MULTIPLIER_DECIMAL_LESS_THAN_ZERO

- Multiplier decimal less than zero.

-4026 COMMISSION_INVALID

- Commission invalid.
- Incorrect profit value.
- %s less than zero.
- %s absolute value greater than %s .

-4027 INVALID_ACCOUNT_TYPE

- Invalid account type.

-4029 INVALID_TICK_SIZE_PRECISION

- Tick size precision is invalid.
- Price decimal precision is incorrect.

-4030 INVALID_STEP_SIZE_PRECISION

- The number of decimal places for the step size is incorrect.

-4031 INVALID_WORKING_TYPE

- Invalid parameter working type: %s

-4032 EXCEED_MAX_CANCEL_ORDER_SIZE

- Exceeds the maximum order quantity that can be canceled.
- Invalid parameter working type: %s

-4044 INVALID_BALANCE_TYPE

- The balance type is incorrect.

-4045 MAX_STOP_ORDER_EXCEEDED

- Reached the stop-loss order limit.

-4055 AMOUNT_MUST_BE_POSITIVE

- The quantity must be a positive integer.

-4056 INVALID_API_KEY_TYPE

- The API key type is invalid.

-4057 INVALID_RSA_PUBLIC_KEY

- The API key is invalid.

-4058 MAX_PRICE_TOO_LARGE

- maxPrice and priceDecimal too large, please check.

-4060 INVALID_POSITION_SIDE

- Invalid position side.

-4061 POSITION_SIDE_NOT_MATCH

- The order's position direction does not match the user's settings.

-4062 REDUCE_ONLY_CONFLICT

- Invalid or improper reduceOnly value.

-4084 UPCOMING_METHOD

- Method is not allowed currently. Coming soon.

-4086 INVALID_PRICE_SPREAD_THRESHOLD

- Invalid price spread threshold.

-4087 REDUCE_ONLY_ORDER_PERMISSION

- Users can only place reduce-only orders.

-4088 NO_PLACE_ORDER_PERMISSION

- User cannot place orders currently.

-4114 INVALID_CLIENT_TRAN_ID_LEN

- clientTranId is not valid.
- The customer's tranId length should be less than 64 characters.

-4115 DUPLICATED_CLIENT_TRAN_ID

- clientTranId is duplicated.
- The client's tranId should be unique within 7 days.

-4118 REDUCE_ONLY_MARGIN_CHECK_FAILED

- ReduceOnly Order failed. Please check your existing position and open orders

-4131 MARKET_ORDER_REJECT

- The counterparty's best price does not meet the PERCENT_PRICE filter limit.

-4135 INVALID_ACTIVATION_PRICE

- Invalid activation price.

-4137 QUANTITY_EXISTS_WITH_CLOSE_POSITION

- Quantity must be zero when closePosition is true.

-4138 REDUCE_ONLY_MUST_BE_TRUE

- Reduce only must be true when closePosition is true.

-4139 ORDER_TYPE_CANNOT_BE_MKT

- Order type cannot be a market order if it cannot be canceled.

-4140 INVALID_OPENING_POSITION_STATUS

- Invalid symbol status for opening position.

-4141 SYMBOL_ALREADY_CLOSED

- Trading pair has been delisted.

-4142 STRATEGY_INVALID_TRIGGER_PRICE

- Rejected: Take Profit or Stop order would be triggered immediately.

-4164 MIN_NOTIONAL

- Order notional must be at least 5.0 (unless you select Reduce Only)
- Order notional must be no smaller than %s (unless you choose Reduce Only)

-4165 INVALID_TIME_INTERVAL

- Invalid time interval
- Maximum time interval is %s days

-4183 PRICE_HIGHTER_THAN_STOP_MULTIPLIER_UP

- Limit price cannot be higher than the cap of %s.
- Take-Profit/Stop-Loss price cannot be higher than the cap of %s.

-4184 PRICE_LOWER_THAN_STOP_MULTIPLIER_DOWN

- Price is below the stop price limit.

- Take-Profit/Stop-Loss price must be above the trigger price $\times$ multiplier floor.
- Order price (limit or TP/SL) can't be below %s.